

Capítulo 19:

Argumentos da linha de comandos

Parâmetro	Detalhes
argc	Contagem de argumentos (<i>argument count</i>) – inicializado com o número de argumentos separados por espaços passados ao programa a partir da linha de comandos, bem como o próprio nome do programa.
argv	Vetor de argumentos (<i>argument vector</i>) – inicializado como um <i>array</i> de ponteiros para caracteres (<i>strings</i>) contendo os argumentos (e o nome do programa) que foram fornecidos na linha de comandos.

Seção 19.1: Imprimir os argumentos para um programa e convertê-los para valores inteiros

O código seguinte irá imprimir os argumentos passados ao programa e tentará converter cada argumento num número (do tipo long):

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <limits.h>

int main(int argc, char* argv[]) {
    for (int i = 1; i < argc; i++) {
        printf("O argumento %d e: %s\n", i, argv[i]);
        errno = 0;
        char *p;
        long argument_numValue = strtol(argv[i], &p, 10);
        if (p == argv[i]) {
            fprintf(stderr, "O argumento %d nao e um numero.\n", i);
        }
        else if ((argument_numValue == LONG_MIN || argument_numValue == LONG_MAX)
&& errno == ERANGE) {
            fprintf(stderr, "O argumento %d esta fora do intervalo valido.\n", i);
        }
        else {
            printf("O argumento %d e um numero e o seu valor e: %ld\n",
                i, argument_numValue);
        }
    }
    return 0;
}
```

Seção 19.2: Imprimir os argumentos da linha de comandos

Após receber os argumentos, pode imprimi-los da seguinte forma:

```
int main(int argc, char **argv)
{
    for (int i = 1; i < argc; i++)
    {
        printf("Argumento %d: [%s]\n", i, argv[i]);
    }
}
```

Notas:

1. O parâmetro `argv` também pode ser definido como `char *argv[]`.
2. O argumento `argv[0]` pode conter o próprio nome do programa (dependendo de como o programa foi executado). O primeiro argumento "real" da linha de comandos encontra-se em `argv[1]`, motivo pelo qual a variável de iteração `i` do ciclo é inicializada a 1.
3. Na instrução de impressão, pode utilizar `*(argv + i)` em vez de `argv[i]` — ambas avaliam para o mesmo resultado, contudo a primeira forma é mais explícita/verbosa.
4. Os parênteses retos em redor do valor do argumento ajudam a identificar o seu início e fim. Isto pode ser inestimável caso existam espaços em branco finais, quebras de linha ou outros caracteres especiais no argumento.

Seção 19.3: Utilizar as ferramentas GNU getopt

As opções de linha de comandos para aplicações não são tratadas pela linguagem C de forma diferente dos restantes argumentos. São apenas argumentos que, num ambiente Linux ou Unix, começam tradicionalmente por um hífen (-).

Em ambientes Linux/Unix com a biblioteca `glibc`, pode utilizar as ferramentas `getopt` para definir, validar e analisar (*parse*) facilmente as opções de linha de comandos. Estas ferramentas esperam que as opções estejam formatadas de acordo com as normas da GNU (*GNU Coding Standards*).

O exemplo abaixo demonstra como manipular opções de linha de comandos com as ferramentas GNU `getopt`:

```
#include <stdio.h>
#include <getopt.h>
#include <string.h>

/* Imprime uma descricao de todas as opcoes suportadas */
void usage (FILE *fp, const char *path)
```

```

{
    /* Obtem apenas a ultima parte do caminho */
    const char *basename = strrchr(path, '/');
    basename = basename ? basename + 1 : path;
    fprintf(fp, "Uso: %s [OPCAO]\n", basename);
    fprintf(fp, " -h, --help\t\t"
            "Imprime esta ajuda e sai.\n");
    fprintf(fp, " -f, --file[=NOME_FICHEIRO]\t"
            "Escreve toda a saida num ficheiro (padrao: out.txt).\n");
    fprintf(fp, " -m, --msg=TEXTO\t"
            "Imprime uma mensagem especifica em vez de 'Hello world'.\n");
}

```

```

/* Analisa as opcoes da linha de comandos e imprime a mensagem */
int main(int argc, char *argv[])

```

```

{
    char filename[256] = { 0 };
    char message[256] = "Hello world";
    FILE *fp;
    int help_flag = 0;
    int opt;

```

```

    /* Tabela de todas as opcoes suportadas na sua forma longa.

```

```

    * Campos: name, has_arg, flag, val

```

```

    * `has_arg` especifica se a opcao longa correspondente pode (ou deve) aceitar um
    argumento.

```

```

    * `flag`: se apontar para uma variavel, esta recebe o valor de `val` quando a opcao esta
    presente.

```

```

    * Se `flag` for NULL, `val` e retornado por `getopt_long`.

```

```

    */

```

```

    struct option longopts[] = {
        { "help", no_argument, &help_flag, 1 },
        { "file", optional_argument, NULL, 'f' },
        { "msg", required_argument, NULL, 'm' },
        { 0 }
    };

```

```

};

```

```

while (1) {

```

```

    opt = getopt_long (argc, argv, "hf::m:", longopts, 0);

```

```

    if (opt == -1) {

```

```

        /* Um valor de retorno -1 indica que nao ha mais opcoes */

```

```

        break;
    }

```

```

    switch (opt) {
        case 'h':

```

```

        help_flag = 1;
        break;

```

```

        case 'f':

```

```

            printf ("optarg: '%s'\n", optarg);

```

```

            strncpy (filename, optarg ? optarg : "out.txt", sizeof (filename));

```

```

            filename[sizeof (filename) - 1] = '\0';

```

```

            break;
    }
}

```

```

    case 'm':
        strncpy (message, optarg, sizeof (message));
        message[sizeof (message) - 1] = '\0';
        break;
    case '?':
        usage (stderr, argv[0]);
        return 1;
    default:
        break;
}
}

if (help_flag) {
    usage (stdout, argv[0]);
    return 0;
}

if (filename[0]) {
    fp = fopen (filename, "w");
} else {
    fp = stdout;
}

if (!fp) {
    fprintf(stderr, "Falha ao abrir o ficheiro.\n");
    return 1;
}

fprintf (fp, "%s\n", message);
fclose (fp);
return 0;
}

```

Pode ser compilado com o gcc:

gcc exemplo.c -o exemplo

Suporta três opções de linha de comandos (--help, --file e --msg), todas com versões curtas (-h, -f e -m). As opções de argumento são formatadas como:

- --opcao=valor (para opções longas)
- -ovalor ou -o"valor" (para opções curtas)